EST-852

Clasificación con datos desbalanceados

Juan Zamora O.

Machine Learning

INSTITUTO DE ESTADÍSTICA
IES PUCV

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

Introducción

- Disponibilidad de datos en distintos dominios
  - Redes de comunicación
  - Finanzas
  - Vigilancia
- Técnicas existentes para la extracción de conocimiento han mostrado ser útiles en diversas problemáticas reales
- Existencia de desbalances notorios en la distribución de los datos compromete el aprendizaje y desempeño

- Detección de fallas en sistemas productivos
- Identificación de intrusos en redes de computadores
- Detección automática de comentarios ofensivos en redes sociales
- Detección de tejido con cancer vs sano

Cualquier problema cuyos datos exhiban una distribución desigual de entre sus clases

El Problema de los datos desbalanceados

- La mayoría de los métodos de propósito general en clasificación basan su desempeño una proporción similar de las etiquetas de clase y un igual costo de clasificación por clase

Por ejemplo:

- Dado que la SVM (soft-margin) intenta minimizar el error total, se encuentra inherentemente sesgada hacia la clase mayoritaria
  - Falta de datos representando a la clase minoritaria, también puede generar un desbalance en los vectores soporte representativos

Existen varias fuentes de desbalance en los datos:

- Intrinseco: Resultado de la naturaleza de los datos
- Extrinseco: Resultado de factores variables tales como el tiempo o el almacenamiento
  - Ej. Interrupciones en el funcionamiento de un dispositivo transmisor
- Adicionalmente, el desbalance puede ser denominado relativo el tamaño de la clase minoritaria es proporcional a la mayoritaria
  - Un aumento en esta última, también provoca un aumento de la primera
- En contraste al desbalance relativo, existe el desbalance debido a observaciones raras.
  - Ej. Una enfermedad rara de la que existen pocos pacientes.

Soluciones actuales

Al aplicar métodos estándar o de propósito general sobre conjuntos de datos desbalanceados.

Existen principalmente, dos grandes familias de métodos:

- Basados en muestreo: Buscan balancear la distribución de los datos
- Sensibles al costo (costo asimétrico): Modifican su función objetivo incorporando un costo de clasificación mayor en instancias de la clase minoritaria

Métodos basados en muestreo

- Random oversampling

  Consiste en seleccionar al azar ejemplos de la clase minoritaria y **agregarlos** al conjunto de datos

  - Problema: Ejemplos repetidos pueden producir sobreajuste

- Random undersampling

  Consiste en seleccionar al azar ejemplos de la clase mayoritaria y **removerlos** del conjunto de datos

  - Problema: Pérdida de información

- Informed undersampling: EasyEnsemble, BalanceCascade, basados en KNN.

  El objetivo es resolver el problema de pérdida de información generado por random undersampling.

  Ejemplos de la clase mayoritaria son escogidos de “mejor” manera que en random undersampling.

- Synthetic sampling (Generan datos): **SMOTE** y derivados.

  Se crean ejemplos de manera artificial usando los ejemplos de la clase minoritaria

  Se escogen al azar puntos a partir del KNN de cada punto de la clase minoritaria. Cada uno de estos vecinos es perturbado para generar un nuevo ejemplo.

  Problema: Puede aumentar el traslape entre las clases.

- Adaptive Synthetic sampling: Borderline-SMOTE, ADASYN

  Intenta abordar el problema de traslape que genera en ocasiones SMOTE.

  Identifica puntos claves de la clase minoritaria para luego generar los ejemplos artificiales.

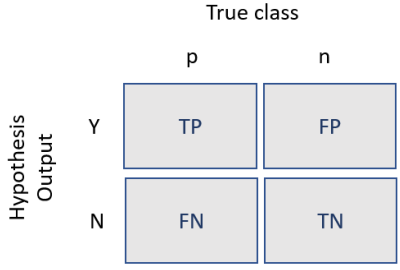
Métodos sensibles al costo

- Consideran el costo asociado con la clasificación errónea de ejemplos
- SVM con costo asimétrico de clasificación [Wang and Japkowicz, 2010]
- GSVM [Gonzalez-Abril et al., 2014]
- One-Class Support Vector Classifier [Li et al., 2006]

Evaluación de desempeño

Es vital contar con métricas para evaluar la efectividad de algoritmos sobre datos desbalanceados.

- Considere un problema de clasificación en dos clases
- Una representación del desempeño en la clasificación puede ser formulado como una matriz de confusión (considere como clase positiva a la minoritaria)



Es frecuente encontrar las siguientes métricas para medir desempeño en la tarea de clasificación:

$$\text{Accuracy} = \frac{TP + TN}{(TP + FN) + (FP + TN)}$$

$$\text{Error rate} = 1 - \text{Accuracy}$$

Existe ciertas situaciones en que este tipo de medidas puee llevar a resultados bastante engañosos.

- Por ejemplo, si del total de los datos, un $\%5$ es de la clases minoritaria.
- Un método que clasifique todo como de la clase mayoritaria tendría Accuracy de $\%95$.

Por lo tanto, la métrica Accuracy **no** provee de información adecuada respecto de la calidad de clasificador $\longrightarrow$ ***Medida sensible a la distribución de los datos***

Otras medidas son adoptadas en lugar de la Accuracy cuando se está en presencia de datos desbalanceados.

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$F_\beta = \frac{(1+\beta^2)\times\text{Recall}\times\text{Precision}}{\beta^2\times\text{Recall}+\text{Precision}}, \text{ usualmente } \beta = 1$$

$$\text{G-mean} = \sqrt{\frac{TP}{FP+FN} \times \frac{TN}{TN+FP}}$$

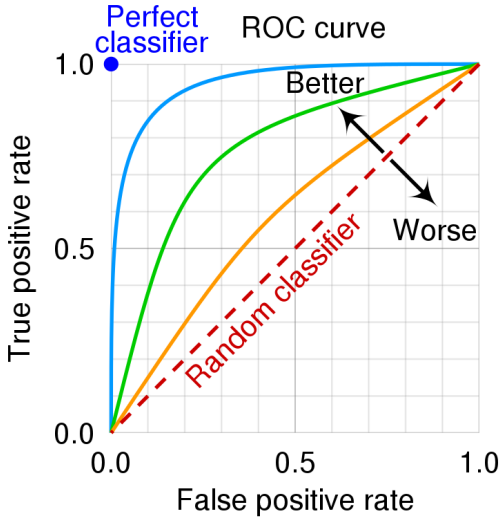
- Precision es una medida de exactitud (de lo detectado, cuánto es correcto)
- Recall es una medidade completitud (cuánto de lo positivo fue detectado correctamente)
- Siempre usar Precision y Recall como parte de un criterio combinado

Receiver Operating Characteristics

- Llamada generalmente Curva ROC [Fawcett, 2006]
- Provee de una representación visual del equilibrio relativo entre el beneficio y el costo de la clasificación, considerando la distribución de los datos.
- Compara dos cantidades extraídas a partir de la matriz de confusión:

$$\text{TP-rate} = \frac{TP}{(TP + FN)} \; ; \; \text{FP-rate} = \frac{FP}{(FP + TN)}$$

- El gráfico ROC consiste en TP-rate vs FP-rate
- El método genera una curva para cada clasificador evaluado



Fuente https://en.wikipedia.org/wiki/Receiver_operating_characteristic

Métodos de ensamblado para clasificación con datos desbalanceados

- Importante paradigma en ML
- Utiliza una colección de clasificadores para realizar predicciones
- Capacidad de generalización generalmente es superior a la de los componentes
- Usados en presencia de desequilibrio de clases para mejorar métodos existentes o crear nuevos

Métodos de ensamblado

- Se entrena un conjunto de aprendices basales (base learners)
- Predicción del ensamblado corresponde a la combinación de las predicciones indiviuales
- Puede mejorar learners bastante frágiles (cercanos al azar) produciendo learners con mayor capacidad de generalización
  - Ej. Árboles de decisión muy superficiales, RNA con una capa oculta y pocas neuronas .

Categorización

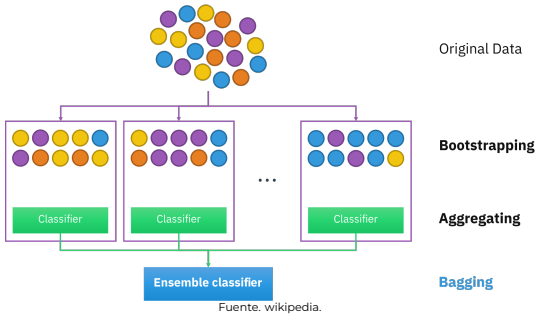
De acuerdo a como se generan los learners, estos métodos se agrupan en 2 paradigmas:

1. Métodos de generación paralela
   - Ejemplo, Bagging
2. Métodos de generación secuencia: Learners anteriores influencian la generación de los subsiguientes.
   - Ejemplo, ADABoost

- Combinación mediante promedio, votación, stacking.

Bagging

- Idea base: Error puede ser reducido al combinar learners independientes.



Fuente. wikipedia.

Algorithm The Bagging algorithm for classification

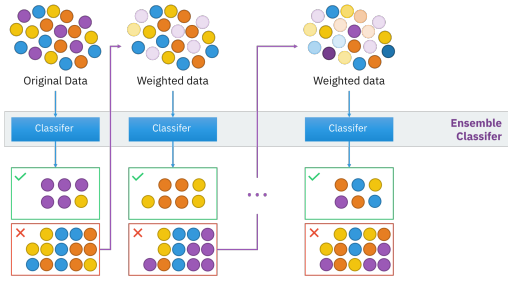
Input: Data set $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$
Base learning algorithm $\mathfrak{L}$
The number of iterations T

1: **for** $t = 1$ to T **do**
2: $h_t = \mathfrak{L}(D, \mathcal{D}_{bs})$ /* $\mathcal{D}_{bs}$ is the bootstrap distribution */
3: **end for**

Output: $H(\mathbf{x}) = \max_y \sum_{t=1}^{T} \mathrm{I}(h_t(\mathbf{x}) = y)$
/* I(x) = 1 if x is true, and 0 otherwise */

Boosting

- Idea base: Corrección progresiva de errores cometidos por learners anteriores.



Fuente. wikipedia.

Algorithm The AdaBoost algorithm

Input: Data set $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$
Base learning algorithm $\mathfrak{L}$
The number of iterations T

1: $\mathcal{D}_1(\mathbf{x}) = 1/n$ /* initialize the weight distribution */
2: **for** $t = 1$ to T **do**
3: $h_t = \mathfrak{L}(D, \mathcal{D}_t)$ /* train h_t from D with distribution $\mathcal{D}_t$ */
4: $\epsilon_t = P_{\mathbf{x}\sim\mathcal{D}_t}(h_t(\mathbf{x}) \neq y(\mathbf{x}))$ /* evaluate the error of h_t */
5: **if** $\epsilon_t > 0.5$ **then**
6: **break**
7: **end if**
8: $\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$ /* Determine the weight of h_t */
9: $\mathcal{D}_{t+1}(\mathbf{x}) = \frac{\mathcal{D}_t(\mathbf{x})}{Z_t} \times \begin{cases} \exp(-\alpha_t) & \text{if } h_t(\mathbf{x}) = y(\mathbf{x}) \\ \exp(\alpha_t) & \text{if } h_t(\mathbf{x}) \neq y(\mathbf{x}) \end{cases}$
$= \frac{\mathcal{D}_t(\mathbf{x})\exp(-\alpha_t f(\mathbf{x}) h_t(\mathbf{x}))}{Z_t}$ /* update the distribution */
10: **end for**

Output: $H(\mathbf{x}) = \texttt{sign}\left(\sum_{t=1}^{T}\alpha_t h_t(\mathbf{x})\right)$

Ensamblado para Datos desbalanceados

- Undersampling: Desperdicio al ignorar información relevante de la clase mayoritaria
- Oversampling: Se puede introducir ruido o traslape entre clases.
  - ¿Que pasaría si en cada ronda de AdaBoost, la distribución de pesos se ajusta mediante SMOTE para aumentar el foco en la clase minoritaria?

Ejemplo: Métodos basados en Boosting

- Con Boosting es posible aumentar el número de ejemplos de clase minoritaria o disminuir el número de la clase mayoritaria
- Permite aumentar el foco en los ejemplos de la clase minoritaria
- Ejemplos: SMOTEBoost y RUSBoost.

SMOTEBoost

- Usa ADABoost como base.
- En cada ronda de boosintg, se usa SMOTE para generar ejemplos sintéticos para la clase minoritaria.
- La distribución de pesos es reajustada
  - Pesos de la clase minoritaria aumentan

Algorithm The SMOTEBoost algorithm

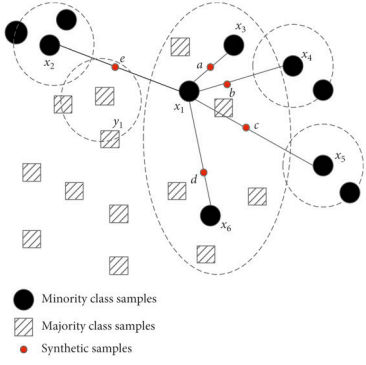
Input:

Data set $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ with $y \in \{1, ..., M\}$, where C_m is the minority class
The number of synthetic examples to be generated in each iteration N
The number of iterations T

1: Let $B = \{(i, y) : i = 1, ..., n, y \neq y_i\}$
2: $\mathcal{D}_1(i, y) = 1/|B|$ for $(i, y) \in B$
3: **for** $i = 1$ to T **do**
4: Modify distribution $\mathcal{D}_t$ by creating N synthetic examples from C_m using the SMOTE algorithm
5: Train a weak learner using distribution $\mathcal{D}_t$
6: Compute weak hypothesis $h_t : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$
7: Compute the pseudo-loss of hypothesis h_t:
$e_t = \sum_{(i,y)\in B} \mathcal{D}_t(i, y)(1 - h_t(\mathbf{x}_i, y_i) + h_t(\mathbf{x}_i, y))$
8: Set $\alpha_t = \ln \frac{1-e_t}{e_t}$
9: Set $d_t = \frac{1}{2}(1 - h_t(\mathbf{x}_i, y) + h_t(\mathbf{x}_i, y_i))$
10: Update $\mathcal{D}_t : \mathcal{D}_{t+1}(i, y) = \frac{\mathcal{D}_t(i,y)}{Z_t} e^{-\alpha_t d_t}$
/* Z_t is a normalization constant such that $\mathcal{D}_{t+1}$ is a distribution */
11: **end for**

Output: $H(\mathbf{x}) = \arg\max_{y\in\mathcal{Y}} \sum_{t=1}^{T} \alpha_t h_t(\mathbf{x}, y)$

SMOTE


x_1
x_2
x_3
x_4
x_5
x_6
y_1
a
b
c
d
e
Minority class samples
Majority class samples
Synthetic samples

Referencias

- Fawcett, T. (2006). An introduction to ROC analysis. Pattern recognition letters, 27(8), 861-874.
- Li, P., Chan, K. L., & Fang, W. (2006, August). Hybrid kernel machine ensemble for imbalanced data sets. In 18th International Conference on Pattern Recognition (ICPR'06) (Vol. 1, pp. 1108-1111). IEEE.
- Wang, B. X., & Japkowicz, N. (2010). Boosting support vector machines for imbalanced data sets. Knowledge and information systems, 25(1), 1-20.
- Gonzalez-Abril, L., Nunez, H., Angulo, C., & Velasco, F. (2014). GSVM: An SVM for handling imbalanced accuracy between classes inbi-classification problems. Applied Soft Computing, 17, 23-31.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: